Ma 3 PS1

Om Sanan

28 January 2026

§1 Gradient Calculation by Hand

We are given

$$L = (w_1 + w_2) \cdot w_3, \qquad s = w_1 + w_2, \qquad L = s \cdot w_3$$

with values

$$w_1 = 1, \quad w_2 = 2.3, \quad s = 3.3, \quad w_3 = 5.7, \quad L = 18.81$$

§1.1 Calculate $\partial L/\partial w_3$

$$\frac{\partial L}{\partial w_3} = \frac{\partial\big((w_1 + w_2)w_3\big)}{\partial w_3} = w_1 + w_2 = s = \boxed{3.3}$$

§1.2 Calculate $\partial L/\partial w_1$ and $\partial L/\partial w_2$ (use chain rule)

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial s} \cdot \frac{\partial s}{\partial w_1} = \frac{\partial (s w_3)}{\partial s} \cdot \frac{\partial (w_1 + w_2)}{\partial w_1} = w_3 \cdot 1 = \boxed{5.7}$$

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial s} \cdot \frac{\partial s}{\partial w_2} = \frac{\partial (s w_3)}{\partial s} \cdot \frac{\partial (w_1 + w_2)}{\partial w_2} = w_3 \cdot 1 = \boxed{5.7}$$

§1.3 Update parameters

$$w_1^* = w_1 - \eta \frac{\partial L}{\partial w_1} = 1 - 0.01(5.7) = \boxed{0.943}$$

$$w_2^* = w_2 - \eta \frac{\partial L}{\partial w_2} = 2.3 - 0.01(5.7) = \boxed{2.243}$$

$$w_3^* = w_3 - \eta \frac{\partial L}{\partial w_3} = 5.7 - 0.01(3.3) = \boxed{5.667}$$

§1.4 Analysis

$$s^* = w_1^* + w_2^* = 0.943 + 2.243 = 3.186$$

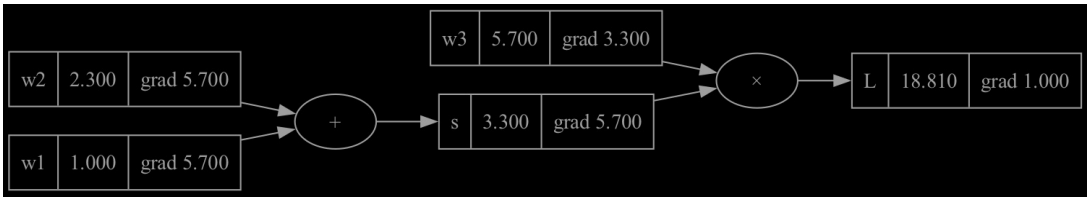
$$L^* = s^* w_3^* = 3.186 \cdot 5.667 = \boxed{18.054062}$$

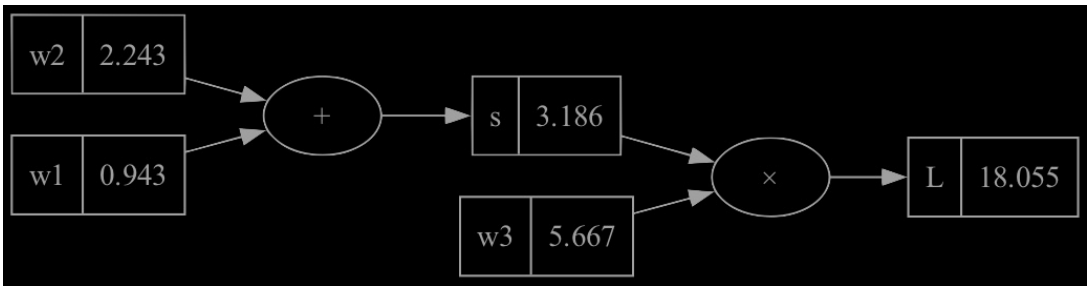
$$L = 18.81, \quad L^* \approx 18.054062 \;\Rightarrow\; \boxed{L \text{ decreased}}$$

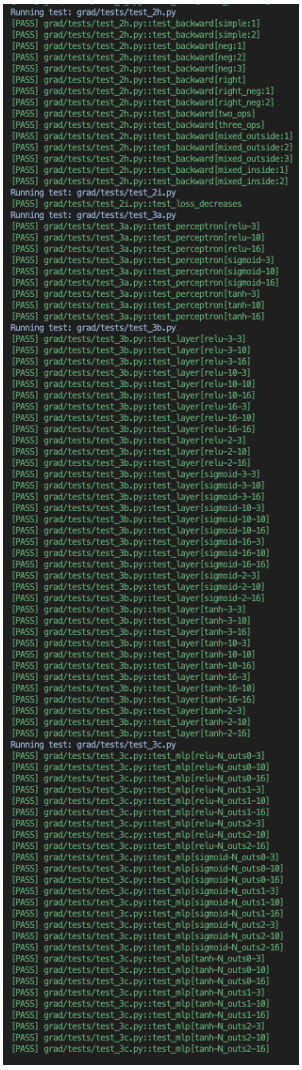
Since gradient descent updates each parameter in the direction opposite ∇L, we expect a small step size $\eta = 0.01$ to reduce the loss, which did happen as we found $L^* < L$.

§2 Autograd from Scratch

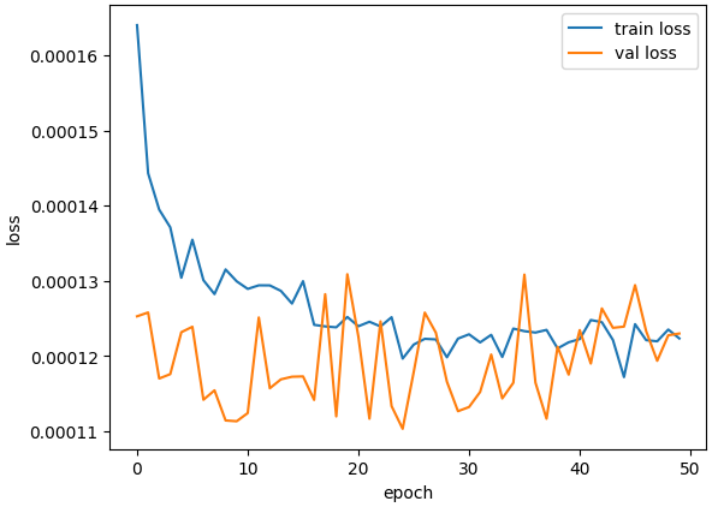
Gradients were same as p1 calculation.

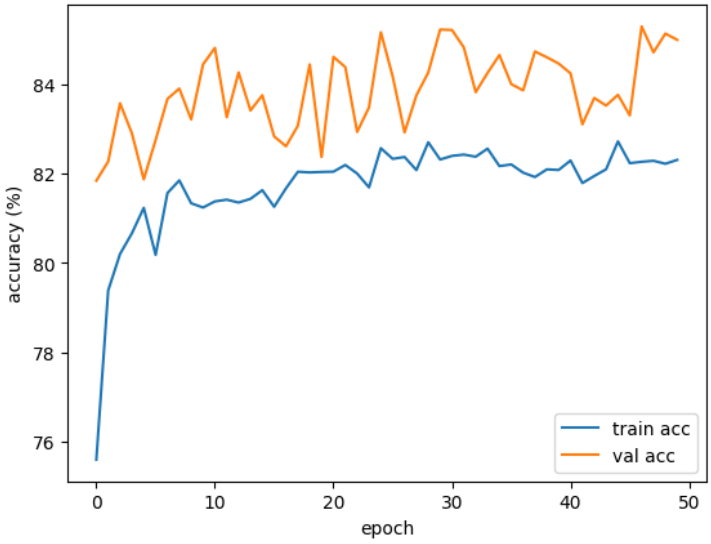






§3 Multi Layer Perceptron (MLP) from Scratch





Epoch 50 — train acc 82.31% — **val acc 85.02%** — train loss 0.0001 — val loss 0.0001

§4 Nonlinearities, loss functions, convergence

§4.1 Show that $\tanh(x) + 1 = 2\sigma(2x)$ where σ is the sigmoid function.

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}, \qquad \sigma(x) = \frac{1}{1 + e^{-x}}$$

$$\tanh(x) + 1 = \frac{e^x - e^{-x}}{e^x + e^{-x}} + 1 = \frac{(e^x - e^{-x}) + (e^x + e^{-x})}{e^x + e^{-x}} = \frac{2e^x}{e^x + e^{-x}}$$

Multiply numerator and denominator by e^x:

$$\tanh(x) + 1 = \frac{2e^{2x}}{e^{2x} + 1} = \frac{2}{1 + e^{-2x}} = 2\sigma(2x)$$

$$\boxed{\tanh(x) + 1 = 2\sigma(2x)}$$

§4.2 Prove that function classes parametrized by both nonlinearities in (1) are identical, which is to say any function representable by a neural network with one activation function can be represented by a network with the other. Consider the role of bias terms.

From part (1),

$$\sigma(2x) = \frac{\tanh(x) + 1}{2} \quad \Longrightarrow \quad \sigma(x) = \frac{\tanh(x/2) + 1}{2}$$

and so,

$$\tanh(x) = 2\sigma(2x) - 1$$

We can then consider a single sigmoid neuron with preactivation $a = w^\top z + b$:

$$h = \sigma(a) = \sigma(w^\top z + b) = \frac{\tanh\left(\frac{w^\top z + b}{2}\right) + 1}{2}$$

Define a tanh neuron output

$$\tilde{h} = \tanh(\tilde{a}), \quad \tilde{a} = \tilde{w}^\top z + \tilde{b}$$

with $\tilde{w} = \frac{1}{2}w$ and $\tilde{b} = \frac{1}{2}b$. Then

$$h = \frac{\tilde{h} + 1}{2} = \frac{1}{2}\tilde{h} + \frac{1}{2}$$

Now consider any next-layer affine map (before the next nonlinearity):

$$a_{\text{next}} = u^\top h + c$$

Substitute $h = \frac{1}{2}\tilde{h} + \frac{1}{2}$:

$$a_{\text{next}} = u^\top\left(\frac{1}{2}\tilde{h} + \frac{1}{2}\right) + c = \left(\frac{1}{2}u\right)^\top \tilde{h} + \left(c + \frac{1}{2}u^\top \mathbf{1}\right)$$

Therefore the scaling by $\frac{1}{2}$ is absorbed into the next-layer weights $u \mapsto \frac{1}{2}u$, and the constant shift $\frac{1}{2}$ is used in the next layer bias $c \mapsto c + \frac{1}{2}u^\top \mathbf{1}$. Applying this conversion at every hidden unit (layer-by-layer) converts a sigmoid network into an equivalent tanh network representing the same function. The reverse direction follows similarly from $\tanh(x) = 2\sigma(2x) - 1$, with constants also absorbed into weights and biases.

$$\boxed{\text{With layers that include bias sigmoid and tanh represent the same function class.}}$$

§4.3 Show that maximizing the log-likelihood function for a model where the outputs represent conditional class probabilities is equivalent to minimizing the cross-entropy loss.

Let the model output class probabilities $p_\theta(y \mid x)$. For data $\{(x_i, y_i)\}_{i=1}^N$, the (conditional) likelihood is

$$\prod_{i=1}^{N} p_\theta(y_i \mid x_i)$$

so the log-likelihood is

$$\log \mathcal{L}(\theta) = \sum_{i=1}^{N} \log p_\theta(y_i \mid x_i)$$

Maximizing $\log \mathcal{L}(\theta)$ is equivalent to minimizing the negative log-likelihood:

$$-\log \mathcal{L}(\theta) = -\sum_{i=1}^{N} \log p_\theta(y_i \mid x_i)$$

For multiclass classification with one-hot labels $y_{ik} \in \{0, 1\}$ and predicted probabilities $p_\theta(y = k \mid x_i) = p_{ik}$,

$$-\sum_{i=1}^{N} \log p_\theta(y_i \mid x_i) = -\sum_{i=1}^{N} \sum_{k} y_{ik} \log p_{ik}$$

which is exactly the cross-entropy loss between the empirical distribution (one-hot) and the model distribution.

$$\boxed{\arg\max_\theta \sum_i \log p_\theta(y_i \mid x_i) \;=\; \arg\min_\theta \Big(-\sum_i \sum_k y_{ik} \log p_{ik} \Big)}$$

§4.4 Provide a justification for using these particular bias correction factors in Adam optimization. You may use words and/or derivations to support your explanation.

Adam uses EMAs of the first and second moments of gradients:

$$s_t = \beta_1 s_{t-1} + (1 - \beta_1) g_t, \qquad r_t = \beta_2 r_{t-1} + (1 - \beta_2) g_t^2, \qquad s_0 = r_0 = 0$$

Unrolling the recursion gives

$$s_t = (1 - \beta_1) \sum_{i=1}^{t} \beta_1^{t-i} g_i$$

If the gradient has stationary mean $\mathbb{E}[g_i] = \mu$, then

$$\mathbb{E}[s_t] = (1 - \beta_1) \sum_{i=1}^{t} \beta_1^{t-i} \mu = \mu(1 - \beta_1^t)$$

so s_t is biased toward 0 by the multiplicative factor $(1 - \beta_1^t)$ due to initialization at zero. Therefore,

$$\bar{s}_t = \frac{s_t}{1 - \beta_1^t} \quad \Longrightarrow \quad \mathbb{E}[\bar{s}_t] = \mu$$

Similarly, with stationary second moment $\mathbb{E}[g_i^2] = \nu$,

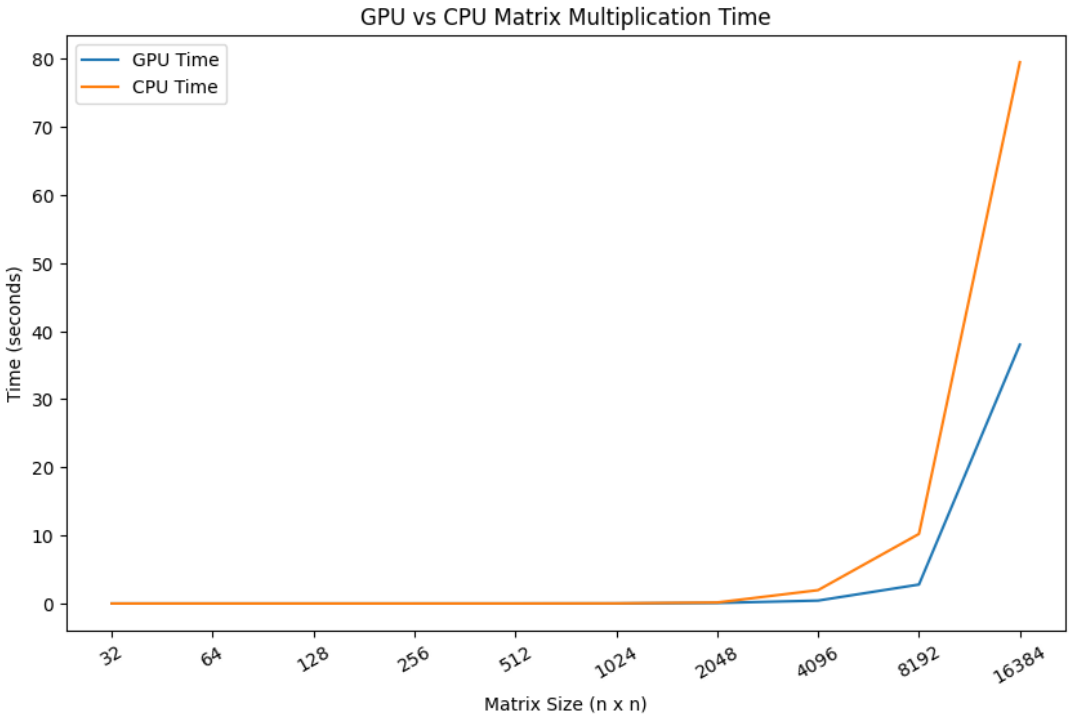
$$\mathbb{E}[r_t] = \nu(1 - \beta_2^t) \quad \Longrightarrow \quad \bar{r}_t = \frac{r_t}{1 - \beta_2^t} \;\text{ is unbiased}$$

This means the bias correction factors $(1-\beta_1^t)$ and $(1-\beta_2^t)$ remove the zero-initialization bias in the early steps, preventing systematically underestimated moment estimates and overly small parameter updates at the start of training.

$$\boxed{\bar{s}_t = \frac{s_t}{1-\beta_1^t}, \quad \bar{r}_t = \frac{r_t}{1-\beta_2^t} \quad \text{correct the zero-init bias of EMAs}}$$

§5 Matrix Multiplication Kernel

1. For n = 2**12 (which is 4096), the doublify kernel executed in 0.001611 seconds.



2. For small matrix sizes (up through 2048), both curves are near zero on this scale which is probably launch overhead + small compute. Starting around 4096, CPU grows to a few seconds, while GPU grows too, but stays lower. This suggests the kernel is likely doing lots of global memory loads repeatedly (no reuse).

3. We can take advantage of our GPU and have a setup where each block loads a tile of A and B into shared memory, then all threads reuse those values to compute the partial dot products. This works because in naive matmul, each A_{ij} and B_{ij} value gets fetched from global memory many times and with tiling, we fetch each value once per tile and reuse it many times from fast shared memory. We can also do memory coalescing for GPU.

§6 Representation power

§6.1 Show that adding layers to a neural network without non-linear activation functions does not increase its expressive power.

Consider a feedforward network with no nonlinearities. Each layer is a map:

$$h^{(1)} = W_1 x + b_1, \quad h^{(2)} = W_2 h^{(1)} + b_2, \quad \ldots, \quad y = W_D h^{(D-1)} + b_D$$

Substitute recursively:

$$y = W_D\big(W_{D-1}(\cdots(W_1 x + b_1) + b_2 \cdots) + b_{D-1}\big) + b_D$$

Collecting terms gives

$$y = \Big(W_D W_{D-1} \cdots W_1\Big)x \; + \; \Big(W_D W_{D-1} \cdots W_2 b_1 + \cdots + W_D b_{D-1} + b_D\Big)$$

Define

$$W := W_D W_{D-1} \cdots W_1, \qquad b := W_D W_{D-1} \cdots W_2 b_1 + \cdots + W_D b_{D-1} + b_D$$

Then the entire deep linear network collapses to a single layer:

$$y = Wx + b$$

This means any function with a depth D linear network is representable by a depth 1 linear network, so adding purely linear layers does not increase expressive power

§6.2 Give an example where adding layers to a deep network without non-linearities could reduce the expressive capacity of the network.

A single linear layer from $\mathbb{R}^2 \to \mathbb{R}^2$ can represent any 2×2 matrix, including maps such as the identity I_2.

Now consider a two-layer linear network constrained to have a width 1:

$$\mathbb{R}^2 \xrightarrow{W_1} \mathbb{R}^1 \xrightarrow{W_2} \mathbb{R}^2$$

The overall linear map is $W = W_2 W_1$, where $W_2 \in \mathbb{R}^{2\times 1}$ and $W_1 \in \mathbb{R}^{1\times 2}$. Therefore

$$\operatorname{rank}(W) = \operatorname{rank}(W_2 W_1) \le \min\{\operatorname{rank}(W_2), \operatorname{rank}(W_1)\} \le 1$$

So this depth 2 network can represent only rank 1 (or rank 0) linear maps, and it cannot represent I_2, which has rank 2. Hence adding a layer with a narrow intermediate width can reduce the set of representable functions

§6.3 Design a one-layer perceptron that fits the logical functions AND and OR for 2D inputs. Why can't a single layer fit XOR?

Let inputs be $x_1, x_2 \in \{0, 1\}$ and a perceptron output be

$$\hat{y} = \mathbf{1}[w_1 x_1 + w_2 x_2 + b \ge 0]$$

For AND, we want $\hat{y} = 1$ only when $(x_1, x_2) = (1, 1)$. Choose

$$w_1 = 1, \quad w_2 = 1, \quad b = -1.5$$

Then

$$(0,0): \; -1.5 < 0, \quad (1,0): \; -0.5 < 0, \quad (0,1): \; -0.5 < 0, \quad (1,1): \; 0.5 \ge 0$$

so this perceptron computes AND

$$\boxed{\text{AND: } w = (1,1),\ b = -1.5}$$

For OR, we want $\hat{y} = 0$ only when $(x_1, x_2) = (0,0)$. Choose

$$w_1 = 1, \quad w_2 = 1, \quad b = -0.5$$

Then

$$(0,0):\ -0.5 < 0, \quad (1,0):\ 0.5 \geq 0, \quad (0,1):\ 0.5 \geq 0, \quad (1,1):\ 1.5 \geq 0$$

so this perceptron computes OR

$$\boxed{\text{OR: } w = (1,1),\ b = -0.5}$$

A single-layer perceptron corresponds to a linear decision boundary which separates the plane into two halves. XOR labels $(0,0)$ and $(1,1)$ as 0 but labels $(1,0)$ and $(0,1)$ as 1, which is not linearly separable. No single line can separate the 1 labeled points from the 0 labeled points, so a one layer perceptron cannot fit XOR.

§6.4 Show that a neural network with a one-dimensional input x and one-dimensional output y, with ReLU activations of depth D and widths W_d, $d = 1, \ldots, D$, can represent piecewise linear functions f with at most $2^D \prod_{d=1}^{D} W_d$ linear pieces.

Let $\kappa(f)$ denote the minimum number of linear pieces needed to represent the piecewise linear function f.

Base case $D = 1$ Any 1-hidden-layer ReLU network with width W_1 has the form

$$f(x) = a_0 + \sum_{j=1}^{W_1} a_j \operatorname{ReLU}(u_j x + v_j)$$

Each $\operatorname{ReLU}(u_j x + v_j)$ is piecewise linear with at most 2 linear pieces (it has one cusp pt where $u_j x + v_j = 0$). A linear combination of such functions yields a piecewise linear function whose breakpoints occur only at those cusp points, so the number of linear pieces is at most the number of distinct breakpoints plus 1, hence at most $W_1 + 1$. In particular, this is $O(W_1)$, and the hint states we can represent any piecewise linear function with N pieces using a width-N one-layer network, so we may take

$$\kappa(f) \leq W_1$$

and since $2^1 \prod_{d=1}^{1} W_d = 2W_1$, the claimed upper bound holds for $D = 1$:

$$\kappa(f) \leq 2W_1$$

Inductive step Assume that after $D - 1$ ReLU layers of widths $W_1, \ldots, W_{D-1}$, every unit output can be written as a piecewise linear function with at most

$$K_{D-1} := 2^{D-1} \prod_{d=1}^{D-1} W_d$$

linear pieces.

Consider layer D with width W_D. Each pre-activation at this layer is an affine combination of previous-layer outputs:

$$g_j(x) = \sum_{i=1}^{W_{D-1}} \alpha_{ji}\, h_i^{(D-1)}(x) + \beta_j$$

By the induction hypothesis each $h_i^{(D-1)}(x)$ has at most K_{D-1} pieces. Using the hint that the number of linear pieces in a sum is at most the sum of the numbers of pieces, scaling does not change the number of pieces, and adding a constant does not change the number of pieces, we have

$$\kappa(g_j) \leq \sum_{i=1}^{W_{D-1}} \kappa(h_i^{(D-1)}) \leq W_{D-1} K_{D-1}$$

Now apply ReLU:

$$h_j^{(D)}(x) = \mathrm{ReLU}(g_j(x))$$

By the hint, applying ReLU can at most double the number of pieces:

$$\kappa(h_j^{(D)}) \leq 2\,\kappa(g_j) \leq 2W_{D-1} K_{D-1}$$

There are W_D such units in layer D, and the output is an affine combination of them, so again by the "sum of pieces" bound,

$$\kappa(y) \leq \sum_{j=1}^{W_D} \kappa(h_j^{(D)}) \leq W_D \cdot 2W_{D-1} K_{D-1}$$

Substitute $K_{D-1} = 2^{D-1} \prod_{d=1}^{D-1} W_d$:

$$\kappa(y) \leq W_D \cdot 2W_{D-1} \cdot 2^{D-1} \prod_{d=1}^{D-1} W_d = 2^D \Big(\prod_{d=1}^{D} W_d \Big)$$

Therefore,

$$\boxed{\kappa(y) \leq 2^D \prod_{d=1}^{D} W_d}$$